Adding HDFE controls to `did_multiplegt_dyn`

Math + `reghdfe`/`hdfe` integration for Stage 1

Running example: `absorb(state#year)` on top of time FEs

1 What we are adding, and why

Stage 1 of `did_multiplegt_dyn` partials *only* time fixed effects (optionally interacted with the time-invariant `trends_nonparam` stratum) out of $\Delta Y_{g,t}$ and each $\Delta X_{k,g,t}$. We want to also partial out an arbitrary, user-supplied **high-dimensional fixed-effect set**, e.g. `absorb(state#year)`, where `state` is a time-invariant group attribute and the interaction `state#year` produces one dummy per (state, year) cell. The HDFE must be partialled out of both $\Delta Y_{g,t}$ and every $\Delta X_{k,g,t}$ *separately for each baseline-treatment level* d, because $\widehat{\boldsymbol{\theta}}_d$ is estimated per d.

Concrete user call we want to make valid:

```
did_multiplegt_dyn Y G T D, effects(4) controls(X1 X2) ///
                  absorb(state#year) cluster(state)
```

where `state` is a time-invariant variable already present at the `(group, time)` level (e.g. each county g has a single `state`).

2 The augmented projector

Let $\mathcal{F}_d$ be the linear span of all dummies absorbed inside baseline cell d:

$$\mathcal{F}_d \;=\; \operatorname{span}\Big\{ \mathbb{1}\{t = t_0\},\;\; \mathbb{1}\{s_g = s_0\}\cdot\mathbb{1}\{t = t_0\},\;\; \mathbb{1}\{A_{g,t} = a_0\} \;:\; t_0, s_0, a_0 \Big\}, \tag{1}$$

restricted to $(g,t) \in \mathcal{S}_d$. Here $A_{g,t}$ ranges over the user's `absorb()` terms (`state`, `state#year`, ...). With `absorb(state#year)` concretely:

$$\mathcal{F}_d \;=\; \operatorname{span}\Big\{ \mathbb{1}\{t = t_0\},\;\; \mathbb{1}\{s_g = s_0\}\cdot\mathbb{1}\{t = t_0\},\;\; \mathbb{1}\{\text{state}_g = a_0\}\cdot\mathbb{1}\{t = t_0\} \Big\}.$$

Notice that `state#year` subsumes the pure time FE – that is fine, the projector handles the collinearity. The point is: the *conditional mean* that we subtract from $\Delta X_{k,g,t}$ and from $\Delta Y_{g,t}$ now varies by $(d, \text{state}_g, t, s_g)$ instead of just (d, t, s_g).

Let $\boldsymbol{W}_d = \operatorname{diag}(N_{g,t})$ on $\mathcal{S}_d$. Define the weighted projector onto $\mathcal{F}_d$ and its annihilator:

$$P^{W}_{\mathcal{F}_d} \;=\; \boldsymbol{Z}_d \left(\boldsymbol{Z}_d^{\top}\boldsymbol{W}_d\boldsymbol{Z}_d\right)^{+} \boldsymbol{Z}_d^{\top}\boldsymbol{W}_d, \qquad M^{W}_{\mathcal{F}_d} \;=\; \boldsymbol{I} - P^{W}_{\mathcal{F}_d}, \tag{2}$$

where $\boldsymbol{Z}_d$ stacks the dummies in (**??**) and $(\cdot)^{+}$ is the Moore–Penrose inverse (to be safe against collinearity among absorb terms).

3 The math, side by side with the old Stage 1

The whole Stage 1 derivation goes through unchanged with one replacement: $P^W_{\mathcal{F}_d}$ subtracts cell means over the richer partition. Below, the *new* formulas; compare against the previous note.

Step A1 – Residualize $\Delta X_{k,g,t}$ and $\Delta Y_{g,t}$ on $\mathcal{F}_d$

For each baseline d and each k:

$$\widetilde{\Delta X}^{(\mathrm{HDFE})}_{k,g,t} \;=\; \sqrt{N_{g,t}}\,\big[M^W_{\mathcal{F}_d}\,\Delta X_k\big]_{g,t}, \tag{3}$$

$$\widetilde{\Delta Y}^{(\mathrm{HDFE})}_{g,t} \;=\; \sqrt{N_{g,t}}\,\big[M^W_{\mathcal{F}_d}\,\Delta Y\big]_{g,t}. \tag{4}$$

When $\mathcal{F}_d$ is just the time FE (current code), $P^W_{\mathcal{F}_d}$ reduces to a weighted within-t mean inside baseline d, which is exactly equation (**??**) in the previous note. With `absorb(state#year)`, $P^W_{\mathcal{F}_d}$ subtracts the weighted within-(state, year, d) mean.

Step A2 – Per-control score for the variance

$$\mathrm{prod\_X}^{(\mathrm{HDFE})}_{k,g,t} \;=\; \sqrt{N_{g,t}}\cdot\widetilde{\Delta X}^{(\mathrm{HDFE})}_{k,g,t} \;=\; N_{g,t}\cdot\big[M^W_{\mathcal{F}_d}\,\Delta X_k\big]_{g,t}. \tag{5}$$

Step A3 – $\widehat{\boldsymbol{\theta}}_d$ via the augmented projection

Stack $\widetilde{\Delta X}^{(\mathrm{HDFE})}_{g,t} \in \mathbb{R}^K$. Then

$$\boxed{\widehat{\boldsymbol{\theta}}^{(\mathrm{HDFE})}_d \;=\; \Big(\textstyle\sum_{(g,t)\in\mathcal{S}_d}\widetilde{\Delta X}^{(\mathrm{HDFE})}_{g,t}\,\widetilde{\Delta X}^{(\mathrm{HDFE})\top}_{g,t}\Big)^{-1}\sum_{(g,t)\in\mathcal{S}_d}\widetilde{\Delta X}^{(\mathrm{HDFE})}_{g,t}\,\widetilde{\Delta Y}^{(\mathrm{HDFE})}_{g,t}.} \tag{6}$$

By Frisch–Waugh–Lovell this equals

$$\widehat{\boldsymbol{\theta}}^{(\mathrm{HDFE})}_d \;=\; \arg\min_{\boldsymbol{\theta},\,\boldsymbol{\alpha}}\sum_{(g,t)\in\mathcal{S}_d} N_{g,t}\,\big(\Delta Y_{g,t} - ^{\top}\boldsymbol{\theta} - \boldsymbol{Z}^{\top}_{d,g,t}\boldsymbol{\alpha}\big)^2, \tag{7}$$

i.e. the weighted OLS of $\Delta Y_{g,t}$ on absorbing the full FE set $\mathcal{F}_d$. This is exactly what `reghdfe` computes.

Step A4 – Inverse-denominator matrix

$$\mathrm{Den}^{-1,\,(\mathrm{HDFE})}_d \;=\; \Big(\textstyle\sum_{(g,t)\in\mathcal{S}_d}\widetilde{\Delta X}^{(\mathrm{HDFE})}_{g,t}\,\widetilde{\Delta X}^{(\mathrm{HDFE})\top}_{g,t}\Big)^{-1}\cdot N^{\mathrm{tot}}_d\cdot G. \tag{8}$$

The scalar N^{tot}_d and G are unchanged.

Step A5 – The fitted FE-only mean of $\Delta Y_{g,t}$

$$\widehat{E}^{(d,\,\mathrm{HDFE})}_{g,t} \;=\; \big[P^W_{\mathcal{F}_d}\,\Delta Y\big]_{g,t} \;=\; \widehat{\mathbb{E}}\Big[\,\Delta Y_{g,t}\,\Big|\,\boldsymbol{Z}_{d,g,t},\ (g,t)\in\mathcal{S}_d\Big]. \tag{9}$$

Built once per d. Used at L4664 and L5210 to demean $\Delta Y_{g,t}$ inside the score in_sum$_{k,d}$. With multi-dim absorb, this is no longer a single `gegen` – it's the prediction from a `reghdfe` of $\Delta Y_{g,t}$ on the absorb set alone.

Step A6 – DoF and singularity bookkeeping

Let $r_d = \text{rank}(\boldsymbol{Z}_d^\top \boldsymbol{W}_d \boldsymbol{Z}_d) =$ `e(df_a)` from `reghdfe`. The singularity guard [ado L1042] extends to

$$\#\mathcal{S}_d \;>\; r_d + K, \qquad \left|\det\!\left(\widetilde{X}^\top \widetilde{X}\right)\right| > 10^{-16}, \tag{10}$$

and the warning at L1086 should additionally name the absorb dimension that is starving the regression.

4 Why `reghdfe` (or `hdfe`) replaces the per-control loop

In the current code, three computations are done by hand inside a `foreach var of varlist ‘controls’` loop:

1. the weighted cell mean of $\Delta X_{k,g,t}$ [ado L931–934];
2. the residual $\widetilde{\Delta X}_k$ [ado L941];
3. the weighted regression of $\Delta Y_{g,t}$ on $\Delta X_{k,g,t}$ + time FEs [ado L984–991] to produce $\widehat{E}^{(d)}_{g,t}$.

The first two are a one-way demean – cheap to do by hand. With HDFE, the projector $P^W_{\mathcal{F}_d}$ is multi-dimensional and cannot be computed with a single `bys ... : gegen`. `hdfe` (and `reghdfe` under the hood) runs the iterative alternating-projections algorithm of Guimarães–Portugal that converges to $P^W_{\mathcal{F}_d}$, with one call demeaning all target variables at once. This is the single conceptual swap.

Two options for the call, both valid:

(O1) `hdfe` **as a demean utility.** Demean $\Delta Y_{g,t}$ and all $\Delta X_{k,g,t}$ in one call, then do *exactly* the current `matrix accum` dance to recover $\widehat{\boldsymbol{\theta}}_d$, Den_d^{-1}, $\widehat{E}^{(d)}_{g,t}$. Minimum disruption to the rest of the code.

(O2) `reghdfe` **as the regression.** Read $\widehat{\boldsymbol{\theta}}_d$ from `e(b)`; read r_d from `e(df_a)`; `predict` fits to recover $\widehat{E}^{(d)}_{g,t}$. Skip `matrix accum`. Simpler but two passes through the data (one for $\widehat{\boldsymbol{\theta}}_d$, one for $\widehat{E}^{(d)}$).

I recommend **(O1)** for the patch: it changes only the demean step (L931–941) and leaves the per-d matrix-accum block (L1018–1052) – which is the variance-critical part – byte-for-byte the same. We never have to trust `reghdfe`'s SE or DoF correction; we keep our own.

5 Annotated Stata patch (option O1)

The block below replaces the body of the `foreach var of varlist ‘controls’` demeaning loop in `did_multiplegt_dyn.ado` around L919–961, plus the FE-only prediction at L984–999. New code is highlighted in comments.

```
* ---- New: parse absorb() ------------------------------------------------
* Add to the syntax line at L53:
*     absorb(string)
* and at the top of the controls block do:

local has_absorb = ("‘absorb’" != "")
if ‘has_absorb’ {
```

```
    qui cap which hdfe
    if _rc {
        di as error "absorb() requires hdfe. Run: ssc install hdfe, replace"
        exit 198
    }
}

* The full FE set (per baseline d): time + trends_nonparam#time + absorb.
* trends_nonparam is interacted with time to match current semantics.
local fe_set "time_XX"
if "`trends_nonparam'" != "" local fe_set "`fe_set' i.trends_nonparam_temp_XX#i.time_XX"
if `has_absorb' local fe_set "`fe_set' `absorb'"

* ---- Replaces L919--961: demean DY and each DX_k by F_d, per baseline ------
local mycontrols_XX ""
local count_controls = 0
foreach var of varlist `controls' {
    local ++count_controls
    capture drop resid_X`count_controls'_time_FE_XX
    capture drop prod_X`count_controls'_Ngt_XX
    local mycontrols_XX "`mycontrols_XX' resid_X`count_controls'_time_FE_XX"
}
capture drop diff_y_wXX
capture drop diff_y_resid_XX

levelsof d_sq_int_XX, local(levels_d_sq_XX)
if "`trends_nonparam'" != "" {
    capture drop trends_nonparam_temp_XX
    gegen trends_nonparam_temp_XX = group(`trends_nonparam')
}

foreach l of local levels_d_sq_XX {

    * Variables to demean in this baseline cell d = `l'.
    local dvars "diff_y_XX"
    forvalues k = 1/`count_controls' {
        local dvars "`dvars' diff_X`k'_XX"
    }

    * Sample: not-yet-switcher control sample, all FDs non-missing.
    tempvar smp
    gen byte `smp' = (ever_change_d_XX == 0 ///
                    & diff_y_XX != .       ///
                    & fd_X_all_non_missing_XX == 1 ///
                    & d_sq_int_XX == `l'   ///
                    & time_XX < F_g_XX)

    * One call: demean every target variable by the full FE set, in place.
    * hdfe writes the demeaned values into resXX_diff_y_XX, resXX_diff_X1_XX, ...
    qui hdfe `dvars' [aw=N_gt_XX] if `smp', ///
        absorb(`fe_set') generate(resXX_)

    * Pack residuals into the variables the rest of the code expects.
    * Multiply by sqrt(N_gt) to match the existing weighted-OLS-on-scaled
```

```
    * variables convention (so matrix accum below gives X'WX directly).
    forvalues k = 1/`count_controls' {
        replace resid_X`k'_time_FE_XX = sqrt(N_gt_XX) * resXX_diff_X`k'_XX ///
            if d_sq_int_XX == `l' & `smp' == 1
        replace resid_X`k'_time_FE_XX = 0 if missing(resid_X`k'_time_FE_XX) ///
            & d_sq_int_XX == `l'
        replace prod_X`k'_Ngt_XX = sqrt(N_gt_XX) * resid_X`k'_time_FE_XX ///
            if d_sq_int_XX == `l'
        replace prod_X`k'_Ngt_XX = 0 if missing(prod_X`k'_Ngt_XX) ///
            & d_sq_int_XX == `l'
        drop resXX_diff_X`k'_XX
    }
    replace diff_y_wXX = sqrt(N_gt_XX) * diff_y_XX if d_sq_int_XX == `l'

    * ---- Replaces L984--996: FE-only fitted value of DY, by reghdfe ------
    * E_y_hat_gt_int_l_XX = P_F_d * DY, the projection of DY onto F_d alone.
    cap drop E_y_hat_gt_int_`l'_XX
    qui reghdfe diff_y_XX [aw=N_gt_XX] if `smp', ///
        absorb(`fe_set') residuals(resid_DY_l_XX)
    * Fitted value = DY - residual:
    gen E_y_hat_gt_int_`l'_XX = diff_y_XX - resid_DY_l_XX ///
        if d_sq_int_XX == `l' & time_XX < F_g_XX
    drop resid_DY_l_XX

    * Track absorbed-rank for the singularity guard at L1042.
    scalar df_a_`l'_XX = e(df_a)

    drop `smp'

    * ---- L1018 onward (matrix accum, theta_d, inv_Denom) is UNCHANGED -----
    * Because resid_X*_time_FE_XX and diff_y_wXX are now built off the
    * HDFE projection, the OLS-on-residuals trick still recovers theta_d.
}
```

A few short remarks on the patch:

- The block *below* L1018 – which builds $\boldsymbol{M}_d$, $\boldsymbol{q}_d$, $\widehat{\boldsymbol{\theta}}_d$, Den_d^{-1} via `matrix accum` – is left unchanged. By FWL, plugging in $M^W_{\mathcal{F}_d}$-residualized inputs gives the right $\widehat{\boldsymbol{\theta}}_d$ for the augmented projection (**??**).
- Stages 2, 3, 4 (the long-difference residualization at L4679 and the $U^{\mathrm{var},X}$ correction at L4905–4954) need *no* code change: they consume `coefs_sq_l_XX`, `inv_Denom_l_XX`, `prod_X*_Ngt_XX`, and `E_y_hat_gt_int_l_XX`. Each of these is now built off the augmented projector, so the variance correction automatically picks up the larger FE set.
- The placebo path at L5145–L5184 also needs no change for the same reason: it just calls `coefs_sq_l_XX` at L5177.
- Add `absorb()` to the inner-call locals at L1337, L2030, L2033 so the bootstrap and `by_path` branches replay it.
- Add `absorb` to `predict_het` incompatibility check at L457–464 (since it implies controls are present).

6 Sample alignment, weights, missing values

A few things to double-check when implementing:

1. **Sample.** `hdfe` restricts to `if` and observations with non-missing weights. We pass `'smp' == 1` which already includes `fd_X_all_non_missing_XX == 1`, matching the current code's behavior [ado L931].
2. **Weights.** `hdfe` accepts `[aw=N_gt_XX]`. Internally it uses weighted means in each FE block; the resulting residuals satisfy $\boldsymbol{W}_d^{1/2} M_{\mathcal{F}_d}^{W} \boldsymbol{W}_d^{1/2}$ symmetry, so scaling by $\sqrt{N_{g,t}}$ afterwards reproduces the current $\widetilde{\Delta X}_k$ convention exactly when $\mathcal{F}_d = \text{span}\{\mathbb{1}\{t = t_0\}\}$.
3. **Missingness.** Each $\Delta X_{k,g,t}$ is missing in period $t = 1$; the flag `fd_X_all_non_missing_XX` already drops these. For $t \geq 2$ but $\Delta X_{k,g,t}$ still missing (gaps in the panel), `hdfe` drops them automatically – the sample restriction `'smp'` catches them via `fd_X_all_non_missing_XX`.
4. **Collinearity inside `absorb()`.** If the user passes `absorb(state#year)` alongside our forced `time_XX`, the year dummies are absorbed twice; `hdfe` handles this gracefully via the Moore–Penrose-style alternating projection. Just be sure `e(df_a)` from `reghdfe` correctly reflects the absorbed rank net of collinearity, then the singularity guard (`??`) fires correctly.
5. **Per-baseline call.** `hdfe` is called once per baseline cell d because $\mathcal{F}_d$ is restricted to $\mathcal{S}_d$. With many baseline levels this is the main performance cost; for `|absorb|=0`, we keep the fast `gegen` path unconditionally to avoid the overhead.

7 What the user gets at the command line

```
* time-invariant state attribute, e.g. each county g has a state
* Before:  controls(X1 X2) trends_nonparam(state)
*   -> partials state * year FEs out of DX/DY (one stratum per state).
* After:  controls(X1 X2) absorb(state#year)
*   -> partials state * year FEs out of DX/DY (one dummy per (state, year)).
*      Equivalent in this simple case, but absorb() also supports
*      multi-way: absorb(state#year industry#year), and finer cuts
*      that trends_nonparam cannot express (e.g. county FE in FD).
did_multiplegt_dyn Y G T D, effects(4) placebo(2) ///
                  controls(X1 X2) absorb(state#year) cluster(state)
```

The estimated $\widehat{\boldsymbol{\theta}}_d$ now solves

$$\min_{\boldsymbol{\theta},\, \alpha^{(d)}_{\text{state}\times\text{year}}} \sum_{(g,t)\in\mathcal{S}_d} N_{g,t} \left(\Delta Y_{g,t} - {}^{\top}\boldsymbol{\theta} - \alpha^{(d)}_{\text{state}_g,t} \right)^2,$$

and the long-difference of Y is residualized as $Y_{g,t} - Y_{g,t-\ell} - (X_{g,t} - X_{g,t-\ell})^{\top}\widehat{\boldsymbol{\theta}}_d$ at L4679, exactly as today.

8 Summary of code touches

Line	File region	Change
L53	syntax	add `absorb(string)`
L66	dependency check	add an `hdfe` check mirroring the `gtools` one
L457–464	`predict_het` guard	also forbid `predict_het` with `absorb()`
L919–961	controls demean	replace with the `hdfe` call shown above
L984–996	$\widehat{E}^{(d)}$ build	replace `reg ... ibn.time_XX` with `reghdfe ... absorb('fe_set')`
L1042	singularity guard	adjust for `e(df_a)` from `reghdfe`
L1086	error message	name the absorb dimension that triggered it
L1337, L2030, L2033	inner-call locals	add `absorb('absorb')`
L4582+ / L5145+	long-diff for effects / placebos	*no change* – consume the already-augmented $\widehat{\boldsymbol{\theta}}_d$

End of patch. The math is a one-line replacement of the projector; the code is one `hdfe` call (plus one `reghdfe` for $\widehat{E}^{(d)}$) per baseline-treatment cell.